

Platform Technical Documentation

The operating system for business transformation

Complete architecture, data model, roles, features, security & operations.
Prepared as a knowledge base for training an AI project on the platform.

Stack: TanStack Start · React 19 · Supabase (Postgres + Edge Functions) · Tailwind v4
Document version 1.0

1. Platform Overview

VK Mentorship (VKM) is a premium coaching, learning and business-transformation platform for the Venu Kalyan Mentorship community. It runs a structured multi-week program where each participant grows their real business with the help of an assigned Growth Coach, a weekly group class from the mentor (Venu Kalyan), an AI Business Advisor, habit tracking, recognition (points/leaderboard/milestones), and a member community.

The platform is a single application with **role-based experiences**. The same codebase serves four roles, each with its own navigation, dashboards and permissions:

Participant

Coach

Mentor

Super Admin

Core value loop

1. **Mentor** teaches the weekly "why" in a live Tuesday class.
2. **Participant** applies the week's task to their own business and submits proof.
3. **Coach** reviews the proof, awards points, logs coaching notes, drives implementation 1:1.
4. **System** tracks progress, business numbers, habits, and recognition — feeding dashboards for everyone.
5. **Super Admin** runs operations: users, batches, the program plan, providers, security.

2. Technology Stack

Layer	Technology	Notes
Framework	TanStack Start (file routes, SSR, server functions)	Built as a static SPA (prerendered shell) for cPanel/Cloudflare hosting.
UI	React 19, TypeScript, Tailwind CSS v4	Radix UI primitives, Framer Motion, custom "VKM" component kit.
Build	Vite 8, Nitro (node-server preset), Lightning CSS	`npm run build` → `.output/public` static bundle + SPA post-build.
Backend	Supabase: Postgres, Row-Level Security, Auth, Storage	All app data + auth. RLS is the primary authorization mechanism.
Server logic	Supabase Edge Functions (Deno) + TanStack server functions	3 edge functions: <code>messaging</code> , <code>zoom</code> , <code>storage</code> .
Realtime	Supabase Realtime channels	Chat, DMs, notifications, proof queue, habit feed.
Video calls	Zoom Meeting SDK (in-app) via Server-to-Server OAuth	Coach↔participant meetings without leaving the app.

Key constraint: the Supabase *service-role* / secret keys are server-only — never `VITE_`-prefixed, never in the client bundle. Only the publishable (anon) key ships to the browser. API keys for email/SMS/Zoom/AI/storage live in the `messaging_settings` table and are used only inside edge functions.

3. Architecture & Request Flow

Two execution surfaces

- **Client (browser):** reads/writes most data *directly* via the Supabase JS client using the publishable key. Every such call is constrained by Row-Level Security in Postgres.
- **Server (edge functions + TanStack server functions):** used when an action needs the service role or a provider secret — sending email/OTP, creating Zoom meetings, presigning storage uploads, admin user operations (invite, password reset, impersonate, bulk coach assign).

Auth

- Supabase Auth (email/password, email-OTP, and native Google OAuth).
- First user becomes `super_admin` automatically. Roles live in `user_roles` (enum `app_role`: participant/coach/mentor/super_admin).
- Invited users get a temp password and must reset on first login (`profiles.must_reset_password`).
- The client holds the session; `useAuth()` exposes `user, session, profile, roles, primaryRole` .

Deployment model

- `npm run build` produces `.output/public` — a prerendered SPA shell + hashed assets. A post-build step (`scripts/spa-postbuild.mjs`) rewrites stale asset references so Apache/cPanel serves it correctly.
- `public/.htaccess` provides SPA routing (fallback to `index.html`), long-cache for assets, and security headers (CSP, HSTS, X-Frame-Options, nosniff, Referrer-Policy, Permissions-Policy).
- `.npmrc` sets `legacy-peer-deps=true` so CI installs past the Zoom SDK's React-18 peer pin (the app runs React 19).

4. Roles & Access Model (RBAC)

Role	Sees	Can do
Participant	Only their own data.	Run their program, submit proofs, log habits/focus, track business numbers + team, set vision/goals, use AI advisor, chat with coach, join the community.
Coach	Only <i>assigned</i> participants.	Review proofs, award points, write coaching notes, manage coach tasks, see cohort overview & performance, message assigned participants, run meetings.
Mentor	Org-wide oversight.	Everything a coach sees but across all participants; manage programs/batches; broadcast.
Super Admin	Everything.	User management, program builder, batches, storage & messaging providers, security, all operations.

5. Security Model (the heart of the platform)

Authorization is enforced in the database with **Row-Level Security (RLS)**, not in the UI. The UI only reflects what the database will allow. Several `SECURITY DEFINER` helper functions centralize the rules:

Helper	Returns true when...
<code>has_role(uid, role)</code>	the user has that role in <code>user_roles</code> .
<code>is_staff()</code>	caller is coach OR mentor OR super_admin.
<code>coaches_participant(participant)</code>	caller is super_admin OR mentor (org-wide) OR a coach directly assigned to that participant via <code>coach_assignments</code> .
<code>my_participants()</code>	the set of participant ids the caller may see (assignment-scoped for coaches; all for mentor/admin).

One switch re-scopes the whole coach surface. Almost every participant-data table's read policy is `own row OR super_admin OR coaches_participant(user_id)` . Because of this, redefining `coaches_participant()` instantly re-scopes ~20 tables, the proof queue, habit feed, chat, notes and notifications at once.

Coach assignment

- `coach_assignments` maps one **coach per participant** (reassignable). Only mentor/super_admin can write it.
- A coach therefore sees **only assigned participants**; mentor/super_admin keep org-wide oversight.
- Admins assign/reassign in *User Management* — inline per row, in the user detail drawer, or in bulk (select many → assign a coach).

Other guardrails

- `coaching_notes` is staff-only (participants can never read them); scoped to the coach's assigned participants.
- `coach_tasks` is owner-only.

- Invite acceptance (`accept_invite_by_token`) is bound to the signed-in email — a leaked token can't escalate a different account.
- Untrusted links (proof URLs, notification links) pass through a `safeHref()` allow-list (http/https/mailto only) and render with `rel="noopener noreferrer nofollow"` to block stored-XSS via `javascript:` / `data:` schemes.
- Edge functions support an `ALLOWED_ORIGINS` CORS allow-list and return generic errors on pre-auth paths.
- Invite tokens use 256-bit CSPRNG entropy.

6. Data Model (42 tables)

Grouped by domain. Every participant table is keyed on the owner's `user_id` and read-scoped by the security helpers above.

IDENTITY & ORG

`profiles` (display name, avatar, phone, must_reset flag) · `user_roles` · `user_invites` · `batches` · `batch_members` (cohort membership; role per batch) · `coach_assignments` (participant→coach).

PROGRAM

`programs` (title, duration_weeks, status) · `program_weeks` (the editable week plan: phase, topic, mode, why, task, proof) · `program_enrollments` (per-user `started_at` + `total_weeks` + status — each participant's clock) · `program_settings` · `weekly_progress` (per-week attendance, task, proof, status, points, coach review) · `onboarding_tasks` · `milestones` / `milestone_awards` .

BUSINESS

`business_brains` (the participant's business profile + AI context) · `business_snapshots` (monthly self-reported numbers: revenue, MRR, leads, deals, closing rate, NPS... coach-reviewed) · `monthly_results` · `team_members` (per-business roster: name, role, department, salary, status).

HABITS & ENGAGEMENT

`habit_logs` · `daily_steps` · `daily_water` / `water_events` · `workout_logs` · `focus_sessions` (deep-work timer) · `daily_actions` (today's checklist).

RECOGNITION

`points_ledger` (every point awarded, with source) · leaderboard is computed from it via the `get_leaderboard` function.

VISION

`vision_statements` (5-year north star + mood board) · `vision_goals` (dated goals across years/pillars).

COMMUNITY & COMMS

`member_profiles` (opt-in public directory) · `dm_threads` / `dm_messages` (peer 1:1) · `conversations` / `messages` (participant↔coach support chat) · `notifications` · `meetings` (Zoom) · `coach_visits` · `coaching_notes` · `coach_tasks` · `ai_advisor_threads` .

ADMIN / CONFIG

`messaging_settings` (provider config + secrets for email/SMS/WhatsApp/Zoom/storage; super-admin RLS) · `message_templates` .

7. Program Model (per-user clock + variable length)

- **Each participant starts their own clock** by pressing *Start my program* on the Program Progress page. Day 1 = their start day (`program_enrollments.started_at`); a new week unlocks every 7 days. People who join mid-week start fresh at Week 1.
- **Program length is variable.** It equals the number of `program_weeks` rows for the active program — 14, 16, 18, 19... weeks. Changing the plan applies to everyone, each from their own start date.
- The participant Program Progress page reads the plan from the DB via a `useProgramPlan()` context (built-in 16-week default as fallback until seeded), and uses the live length everywhere (progress bar, "Week X of N", phases, curriculum).

Admin Program Builder (Admin → Academy → Program Builder)

- **Load default** seeds the active program from the built-in 16-week curriculum.
- **Add week** (appends N+1), **Edit** any week (phase, mode, topic, why, task, proof), **Delete** any week (an atomic RPC `admin_delete_program_week` rennumbers the rest so weeks stay contiguous and syncs enrollment lengths).
- Edits reflect for every participant on their next load.

8. Feature Catalogue by Role

8.1 Participant

Area	What it does
Dashboard	Greeting, current week + % progress ring, points/stage/milestones, Today's Focus (real daily_actions), real Revenue/Leads/Closing KPIs from latest snapshot with sparklines, live cohort leaderboard (top 5 + you), milestones, AI nudge, Tuesday-class banner. <i>All real-time.</i>
Program Progress	Start gate → weekly tasks, journey timeline, curriculum. Submit weekly proof (link + files + note) for coach review.
My Business	Cockpit: profile, monthly numbers (revenue, MRR, leads, deals, closing rate, NPS) with trends, analytics, AI insight. Team section: headcount/active/departments/payroll KPIs + full team roster CRUD (name, role, department, salary, status, joined date, contact).
Habits	Daily habit grid, steps, water, workouts — with proof and streaks.
Focus	Deep-work timer (focus_sessions) + Today's Focus checklist (daily_actions).
Vision	5-year statement + mood board + dated goals; primary goal surfaces on the dashboard.
Leaderboard / Milestones / Achievements	Points, stage bands, ranks; milestone unlocks (Goal Setter → System Builder → Growth Champion).
AI Advisor	Private AI trained on the participant's Business Brain; capped input; provider configured by admin.
Community	Opt-in public member directory + peer 1:1 DMs (realtime). Imported/invited participants appear automatically.
Chat / Meetings	Support chat with assigned coach; in-app Zoom meetings.

8.2 Coach

Area	What it does
Participants overview	Only assigned participants (weeks done, points, pending proofs, at-risk).
Participant detail	Full drill-in: personal + Business Brain, Team roster , 16/N-week proof review, habits & engagement, vision, coaching notes. Shared by coach, mentor and super-admin.
Proof queue / reviews	Approve/reject proofs (awards points), request changes (keeps pending + notifies), undo. Habit proof feed.
Cohort & performance	Cohort overview (assignment-scoped) and the coach's own performance metrics.
Coaching notes / tasks	Private 1:1 log per participant (staff-only) + personal coach task list (owner-only).
Notifications	Can notify their own assigned participants; broadcast scoped to their batches.

8.3 Mentor

Same surfaces as coach but **org-wide**: sees all participants, manages programs/batches, and can broadcast to the whole cohort. Uses the same [ParticipantDetail](#) view.

8.4 Super Admin

Area	What it does
User Management	Invite users (auto temp password + secure expiring link), CSV import (creates real batches + community profiles), batch filter, search. Per-user drawer: performance, login/activity, last-active; password reset , login-as-member (impersonation via one-time magic link), move batch , assign/reassign coach (inline, drawer, or bulk). Directory shows an Assigned Coach column.
Program Builder	Add/remove/edit the weekly plan; sets program length. (See §7.)

Storage	Choose upload backend: Supabase Storage or Cloudflare R2 (S3-compatible). R2 creds stored server-side; uploads presigned by the <code>storage</code> edge function. (See §9.)
Email / SMS / WhatsApp	Configure providers + templates + OTP login. Email providers: Resend , MailerSend , Amazon SES , Zoho ZeptoMail . Saving a provider enables it; toggle to disable. (See §10.)
Security & ops	System overview, analytics/reports, batches, coach assignment, VK-Ops workflows (admissions, payments, onboarding, welcome kit, gifts, graduation, etc.).

9. Edge Functions & Storage

messaging

Sends Email/SMS/WhatsApp via the configured provider and issues email-OTP login codes. Reads provider secrets from `messaging_settings` with the service role. Actions: `request_otp` (pre-auth), `send_email`, `send_sms`, `send_whatsapp`, `test` (admin). Email providers implemented: Resend, MailerSend, Amazon SES (SigV4), Zoho ZeptoMail.

zoom

Server-to-Server OAuth. `create_meeting` (staff) and `signature` (host/participant) so calls run in-app via the Zoom Meeting SDK. The host-only `start_url` is never exposed to participants.

storage

Chooses the upload backend. `presign` returns either `{provider:'supabase'}` or a Cloudflare R2 S3 SigV4 presigned PUT URL + public URL. `get_config` / `set_config` (super-admin) manage R2 credentials. A unified client helper `uploadToStorage(bucket, path, file)` routes all four upload sites (chat attachments, avatars, vision board, class videos) through it; the R2 secret never reaches the browser.

10. Email / Messaging Providers

Configured in **Admin** → **Email/SMS/WhatsApp**. Pick a provider, paste credentials, Save (auto-enables), then send a test. Secrets are stored in `messaging_settings` (super-admin RLS) and used only by the `messaging` edge function.

Channel	Providers
Email	Resend · MailerSend · Amazon SES · Zoho ZeptoMail
SMS	Twilio · MSG91
WhatsApp	Twilio · Meta Cloud API

ZeptoMail: uses the Send-Mail token (`Zoho-enczapikey ...`) and a data-center field (`in` for India, `com` for global) → `api.zeptomail.<dc>/v1.1/email` . The sending domain must be verified.

11. Points, Stages & Milestones

- **Points** come from approved weekly proofs (+ per week) and daily habits (+ per task), recorded in `points_ledger` .
- **Stages** (bands by total points): Starter → Builder → Operator → Closer → Growth Champion.
- **Milestones** (3): Goal Setter (≈Wk 3), System Builder (≈Wk 6), Growth Champion (≈Wk 14) — each with a real-world reward; recorded in `milestone_awards` .

12. Build & Deployment

```
npm install      # uses .npmrc legacy-peer-deps for the Zoom SDK
npm run build    # → .output/public (static SPA shell + assets)
npx supabase db push  # apply DB migrations
```

```
npx supabase functions deploy messaging # (and zoom, storage)
npx supabase gen types typescript --linked > src/integrations/supabase/types.ts
```

- **cPanel:** upload the contents of `.output/public` to `public_html`. `.htaccess` handles SPA routing + headers.
- **Cloudflare Pages:** serve the same static output (or the node-server build).
- **Env (build-time, baked):** `VITE_SUPABASE_URL`, `VITE_SUPABASE_PUBLISHABLE_KEY`, `VITE_SUPABASE_PROJECT_ID`. Server-only: `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_URL`, etc.

Static-hosting gotcha (resolved): the home route `/` renders the sign-in page *directly* rather than redirecting. A `beforeLoad` redirect on `/` fired during hydration and caused a React #418 hydration mismatch ("This page didn't load") on static hosts. Rendering the page directly keeps the client render matching the prerendered shell.

13. Operational & Security Checklist

Must-do operational items:

- Keep the Supabase **service-role key** server-only; rotate if ever exposed.
- Set **Google OAuth** provider config in the Supabase dashboard for Google sign-in to work.
- Set `ALLOWED_ORIGINS` on the `messaging` / `zoom` / `storage` edge functions to lock CORS.
- For R2: create the bucket + S3 token, enable public access / custom domain, then fill Admin → Storage.
- For email: verify the sending domain with the chosen provider before going live.
- Decide storage-bucket privacy (public vs signed URLs) per data sensitivity.

14. Key Files Reference

Path	Purpose
<code>src/integrations/supabase/client.ts</code>	Browser Supabase client (publishable key).
<code>src/integrations/supabase/client.server.ts</code>	Service-role client (server only).
<code>src/hooks/use-auth.tsx</code>	Auth context: user, roles, profile.
<code>src/components/participant/program-plan-data.ts</code>	<code>useProgramPlan()</code> — DB-backed week plan.
<code>src/components/participant/enrollment-data.ts</code>	Per-user start + variable length.
<code>src/components/business/business-data.ts</code>	Business snapshots + <code>useTeamMembers()</code> .
<code>src/lib/vkm/admin-users.functions.ts</code>	Admin server fns: detail, reset, impersonate, batch, coach assign.
<code>src/lib/storage-upload.ts</code>	Unified Supabase/R2 upload.
<code>src/lib/vkm/program.ts</code>	Built-in default curriculum, points/stages/milestones.
<code>supabase/functions/{messaging, zoom, storage}</code>	Edge functions.
<code>supabase/migrations/*.sql</code>	50 migrations: schema, RLS, RPCs.

15. Glossary

Batch / Cohort — a group of participants (e.g. "Batch 16"). **Business Brain** — the participant's structured business profile that also powers their AI advisor. **Proof** — evidence a weekly task was done, submitted for coach approval. **Snapshot** — a participant's monthly business numbers. **Stage** — points band. **RLS** — Postgres Row-Level Security, the platform's authorization layer. **Assignment** — the participant→coach mapping that scopes coach visibility. **OMM** — "One Minute Meditation"/daily discipline habit referenced in the curriculum.

